Whitening the Active Denoising Loss

Why $\sqrt{\det M}$ was the wrong scale, and what replaced it

Abstract

The passive diffusion loss is whitened: multiplying the score by $\sigma(t)$ gives a target of unit variance at every noise level. The active loss inherited a single scalar whitener, $\sqrt{\det M}$, applied to both the x and η channels. Because $\sqrt{\det M} = \sqrt{v_x}\sqrt{M_{22}} = \sqrt{v_\eta}\sqrt{M_{11}}$, this is the correct whitener for each channel multiplied by a spurious factor, and one scalar cannot whiten two channels of different scale. At $\tau = 0.5$ the η target consequently varied by $515\times$ across t, collapsing to 0.0014 at $t = 10^{-3}$. Replacing $\sqrt{\det M}$ with the per-channel conditional standard deviations $\sqrt{v_x}, \sqrt{v_\eta}$ restores unit-variance targets at every t and improves FID from 6.91 to 4.98 at matched epoch, sample count and sampler. We verify that the change leaves the minimiser — and therefore the learned score — untouched.

1 The common framework

All three losses in this codebase are weighted denoising score matching. Given a forward kernel $z_t \mid z_0 \sim \mathcal{N}(\mu_t, \Sigma_t)$ and noise $d = z_t - \mu_t$, the *conditional* score is

$$s_{\text{true}} \;=\; \nabla_{z_t} \log p(z_t \mid z_0) \;=\; -\,\Sigma_t^{-1} d, \tag{1}$$

and every loss has the form

$$\mathcal{L} \;=\; \mathbb{E}_{t,z_0,d}\Big[\,\lambda(t)\,\big(F - s_{\text{true}}\big)^2\Big], \qquad \lambda(t) > 0. \tag{2}$$

Two facts follow, and the whole note turns on them.

1. **The minimiser does not depend on λ.** For any positive λ depending only on t (and channel), the minimiser of (2) is $F^\star = \mathbb{E}[s_{\text{true}} \mid z_t] = \nabla \log p_t(z_t)$, the marginal score. *Changing the weighting cannot change what the network learns at convergence.*
2. **λ decides everything else.** With finite capacity and finite training, λ determines how error is traded across t and across channels. A λ that collapses at small t tells the optimiser that small-t errors are free.

The useful choice of λ is the one that makes the *target* $\lambda^{1/2} s_{\text{true}}$ have unit variance at every t — i.e. whitening. Then every noise level contributes equally, no region is silently discarded, and the loss is well-conditioned. This is what DDPM's L_{simple} does, and it is the standard FID-optimal choice.

2 The passive loss: whitening done right

For the passive OU process $\mathrm{d}x = -kx\,\mathrm{d}t + \sqrt{2T_e}\,\mathrm{d}W$ the kernel is scalar, $x_t = \mu_t + \sigma_t\varepsilon$ with $\varepsilon \sim \mathcal{N}(0,1)$. Then

$$s_{\text{true}} = -\frac{x_t - \mu_t}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t} \qquad \Longrightarrow \qquad \sigma_t\, s_{\text{true}} = -\varepsilon \sim \mathcal{N}(0,1). \tag{3}$$

Multiplying by σ_t produces a target of *exactly* unit variance at every t. This is the entire content of "$\sigma \cdot \text{score} = -\epsilon$". The code (`train_multigpu.py`) is:

```
noise = torch.randn_like(images)
x_t, score, mean, std = model.forward(images, noise)
true_score = -(x_t - mean) / (std ** 2)
loss = torch.nn.functional.mse_loss(std * score, std * true_score)
```

Here $\lambda(t) = \sigma_t^2$, the target is $-\varepsilon$, and the loss at zero output is exactly 1 for all t. **The passive baseline was always whitened.** Only the active branch was not, which is why the two were never on comparable footing.

> **Discrepancy worth recording.** The non-AMP branch of the same file weights by σ^2 rather than σ:
>
> ```
> loss = torch.nn.functional.mse_loss((std ** 2) * score,
>                                     (std ** 2) * true_score)
> ```
>
> giving $\lambda = \sigma^4$ and a target of $-\sigma\varepsilon$, which is *not* whitened. All reported runs used `--amp`, so the whitened branch is the one that executed and no result is affected; but the two branches should be made to agree.

3 The active kernel

For the active process the kernel is two-dimensional. With $d = (d_x, d_\eta)^\top$ and

$$M \;=\; \begin{pmatrix} M_{11} & M_{12} \\ M_{12} & M_{22} \end{pmatrix}, \qquad \det M = M_{11}M_{22} - M_{12}^2, \tag{4}$$

equation (1) gives the two components explicitly:

$$s_x = -\,\frac{M_{22}d_x - M_{12}d_\eta}{\det M}, \qquad s_\eta = -\,\frac{M_{11}d_\eta - M_{12}d_x}{\det M}. \tag{5}$$

The question is what to multiply these by.

4 The original active loss: a single scalar whitener

The shipped implementation whitens both channels with the single scalar $\sqrt{\det M}$, in `model_cifar10_sde_DDPM.py`, per RGB channel:

```
det = M11 * M22 - M12 * M12
det = torch.clamp(det, min=1e-8)

for channel in range(3):
    Feta = torch.sqrt(1 / det) * (
        -M11 * (eta_ch - c * eta_0_ch)
        + M12 * (rgb_ch - a * rgb_orig_ch - b * eta_0_ch))
    Fx   = torch.sqrt(1 / det) * (
        -M22 * (rgb_ch - a * rgb_orig_ch - b * eta_0_ch)
        + M12 * (eta_ch - c * eta_0_ch))

    scr_eta = torch.sqrt(det) * F_eta_ch
    scr_x   = torch.sqrt(det) * F_rgb_ch

    loss_eta_ch = torch.mean((scr_eta - Feta) ** 2)
    loss_x_ch   = torch.mean((scr_x   - Fx)   ** 2)
    total_loss += self.Ta * loss_eta_ch + self.Tp * loss_x_ch

total_loss = total_loss / 3.0
```

Reading off the structure: the residuals are $(d_x - ax_0 - b\eta_0) = d_x$ and $(\eta_t - c\eta_0) = d_\eta$, so `Feta` $= \sqrt{\det M}\, s_\eta$ and `Fx` $= \sqrt{\det M}\, s_x$ by (5). The loss is therefore

$$\mathcal{L}_{\text{score}} = T_a\, \mathbb{E}\big[\det M\,(F_\eta - s_\eta)^2\big] + T_p\, \mathbb{E}\big[\det M\,(F_x - s_x)^2\big], \qquad \lambda_\eta = T_a \det M, \;\; \lambda_x = T_p \det M. \tag{6}$$

4.1 Why $\sqrt{\det M}$ is the wrong scale

Let v_x and v_η be the *conditional* variances of the noise components,

$$v_x \;\equiv\; \mathrm{Var}(d_x \mid d_\eta) \;=\; M_{11} - \frac{M_{12}^2}{M_{22}} \;=\; \frac{\det M}{M_{22}}, \qquad v_\eta \;\equiv\; \mathrm{Var}(d_\eta \mid d_x) \;=\; \frac{\det M}{M_{11}}. \tag{7}$$

(Note these are variances of the *noise increments*, not of the data x.) Rearranging (7) gives the identity that explains everything:

$$\boxed{\sqrt{\det M} \;=\; \sqrt{v_x}\,\sqrt{M_{22}} \;=\; \sqrt{v_\eta}\,\sqrt{M_{11}}} \tag{8}$$

The same number factorises two different ways. In the x channel it is the correct whitener $\sqrt{v_x}$ times a spurious $\sqrt{M_{22}}$; in the η channel it is $\sqrt{v_\eta}$ times a spurious $\sqrt{M_{11}}$. A single scalar cannot whiten two channels whose natural scales differ.

The consequence is a target whose scale swings wildly with t. Since $\mathrm{Var}(\sqrt{\det M}\, s_\eta) = M_{11}$ and $\mathrm{Var}(\sqrt{\det M}\, s_x) = M_{22}$ exactly (the $\det M$ cancels), the target standard deviations at $\tau = 0.5$, $k = 4$, $T_a = 6.4$, $T_p = 10^{-3}$ are:

Note that $\det M$ is not itself a mistake — it is damage control. Dropping it entirely would leave a raw target of magnitude ~ 700 under unit weight. The problem is specifically that one scalar is being asked to do two channels' work.

t	η target sd $= \sqrt{M_{11}}$	x target sd $= \sqrt{M_{22}}$	`cond` (both)
0.001	1.417×10^{-3}	2.261×10^{-1}	1
0.010	5.961×10^{-3}	7.084×10^{-1}	1
0.050	4.231×10^{-2}	1.523	1
0.100	1.054×10^{-1}	2.054	1
0.500	5.326×10^{-1}	3.327	1
1.000	6.964×10^{-1}	3.545	1
2.000	7.298×10^{-1}	3.577	1
span	**515**×	15.8×	1×

Table 1: Target scale under the original whitener. The η target collapses by a factor of 515 between $t = T$ and $t = 10^{-3}$; squared, that is a 2.7×10^5 swing in the loss contribution. Small t is effectively invisible to the optimiser.

5 The replacement: per-channel conditional whitening

Whiten each channel by its own conditional standard deviation. The relevant one-dimensional formula is that the i-th score component is the regression residual of d_i on d_j, divided by its conditional variance:

$$s_x = -\frac{d_x - (M_{12}/M_{22})\, d_\eta}{v_x}, \qquad s_\eta = -\frac{d_\eta - (M_{12}/M_{11})\, d_x}{v_\eta}. \tag{9}$$

These are *identical* to (5) — substitute $v_x = \det M / M_{22}$ and the M_{22} cancels — but written in a form where the natural whitener is manifest. Since $\mathrm{Var}\big(d_x - (M_{12}/M_{22}) d_\eta\big) = v_x$, we have $\mathrm{Var}(\sqrt{v_x}\, s_x) = 1$ at every t. The code (opt-in via `--score_param cond`) is:

```
if self.score_param == "cond":
    dx = noise[:, [0, 2, 4]]
    de = noise[:, [1, 3, 5]]
    M11 = M[:, 0, 0].view(-1, 1, 1, 1)
    M12 = M[:, 0, 1].view(-1, 1, 1, 1)
    M22 = M[:, 1, 1].view(-1, 1, 1, 1)
    v_x = torch.clamp(M11 - M12*M12/M22, min=1e-20)  # Var(d_x | d_eta)
    v_e = torch.clamp(M22 - M12*M12/M11, min=1e-20)  # Var(d_eta | d_x)
    s_x_true = -(dx - (M12 / M22) * de) / v_x
    s_e_true = -(de - (M12 / M11) * dx) / v_e
    return (self.Tp * (v_x * (F_x - s_x_true) ** 2).mean()
          + self.Ta * (v_e * (F_eta - s_e_true) ** 2).mean())
```

so

$$\mathcal{L}_{\text{cond}} = T_a\, \mathbb{E}\big[v_\eta\, (F_\eta - s_\eta)^2\big] + T_p\, \mathbb{E}\big[v_x\, (F_x - s_x)^2\big], \qquad \lambda_\eta = T_a v_\eta, \;\; \lambda_x = T_p v_x. \tag{10}$$

This is the exact two-dimensional analogue of the passive $\sigma \cdot \text{score} = -\epsilon$.

The network output is unchanged: it still emits the score. Only the loss weighting differs, so `_score_from_output` is the identity in this mode and sampling is untouched.

5.1 What actually changed

Comparing (6) and (10), the weight ratio is

$$\frac{\lambda_\eta^{\text{cond}}}{\lambda_\eta^{\text{score}}} = \frac{v_\eta}{\det M} = \frac{1}{M_{11}}, \qquad \frac{\lambda_x^{\text{cond}}}{\lambda_x^{\text{score}}} = \frac{1}{M_{22}}. \tag{11}$$

t	0.001	0.01	0.1	1.0	2.0
η weight ratio $1/M_{11}$	4.98×10^5	2.81×10^4	90.0	2.06	1.88

A 2.65×10^5 swing, concentrated at small t — exactly the region the old whitener suppressed. Under `cond` every t contributes equally; under `score` the loss was dominated almost entirely by large t.

5.2 Loss values are not comparable across parameterisations

At zero network output the loss equals the mean squared target, which is known in closed form and differs between the two:

objective	loss at zero output	measured
`cond`	$T_p + T_a = 6.401$	6.403
`score`	$T_a\,\mathbb{E}_t[M_{11}] + T_p\,\mathbb{E}_t[M_{22}] = 2.504$	2.532
passive	1	—

A raw `cond` loss of 0.55 and a raw `score` loss of 0.068 are therefore 8.6% and 2.7% of target variance unexplained — a ratio of 3, not the 8 the raw numbers suggest. Compare only the normalised quantity $\mathcal{L}/\mathcal{L}_0$, and even then remember that the two are fitting differently weighted targets. Training loss is in any case a poor progress signal here: it saturates within ~ 50 epochs while FID continues to improve for thousands.

6 Verification

Both objectives were checked against the shipped classes (`redteam_cond.py`, `redteam_cond_weights.py`).

check	result
`cond` target vs $-\Sigma^{-1}d$, x channel	$\max\lvert\Delta\rvert/\text{RMS} = 5.9 \times 10^{-15}$
`cond` target vs $-\Sigma^{-1}d$, η channel	3.0×10^{-15}
$\mathcal{L}_{\text{score}}$ evaluated at the exact score	2.8×10^{-14}
$\mathcal{L}_{\text{cond}}$ evaluated at the exact score	1.2×10^{-13}
`_score_from_output`, both modes	bit-for-bit identity
v_x in `float32` vs `float64`	4.5×10^{-7}
original `score` branch vs pre-change commit	no deletions

Both losses vanish at the same F, confirming the general argument of Section 1: the minimiser is unchanged, so the network converges to the same score function and the forward dynamics are untouched. Only the finite-capacity, finite-training trade-off moved.

The subtraction in $v_x = M_{11} - M_{12}^2/M_{22}$ was the expected numerical weak point, given that the covariance itself required Van Loan's method for exactly this reason. It is safe here: the x–η correlation peaks at 0.854, well away from 1, so M_{12}^2/M_{22} never approaches M_{11}.

7 Result

Two arms at $\tau = 0.5$, identical in architecture, optimiser, EMA, batch size, schedule and numerics, differing only in `--score_param`:

epoch	measurement	`cond`	`score`
800	$N = 10{,}000$, quadratic PF-300	**7.71**	9.70
800	$N = 50{,}000$, quadratic PF-500	**5.67**	—
1000	$N = 50{,}000$, quadratic PF-500	**4.98**	6.91

A 28% FID reduction at matched epoch, sample count and sampler. The improvement is also faster, not merely offset: over $800 \to 1000$ the `cond` ratio is 0.878 against 0.917 for `score`, and every other per-200-epoch ratio recorded in this project lies in 0.90–0.97.

8 Caveat

Equal weighting buys accuracy at small t, and FID is precisely the metric that rewards small-t fidelity. Nothing here establishes that `cond` is superior on likelihood: the variance-weighted objective (6) is closer in spirit to the ELBO, and the literature consistently finds that L_{simple}-style whitening improves FID while *degrading* NLL relative to likelihood weighting. The claim supported by these measurements is a FID claim, and the weighting should be stated explicitly alongside it.